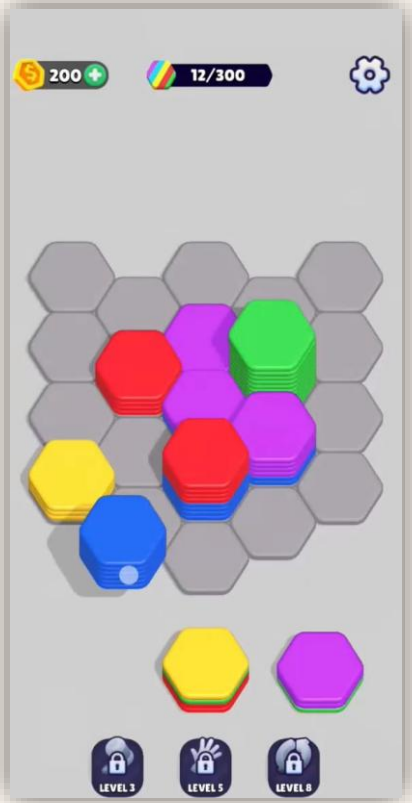
Automated Hexasort Puzzle Creator

My Little Bookstore

Arts and Big Data

Film, TV and Multimedia
2021311066 Shim Ahyeong

1. Project Background



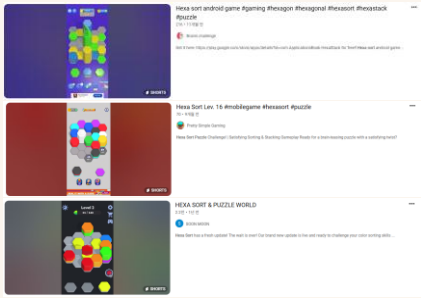
#1 Hexasort Puzzle in Market

My Little Bookstore

Hexasort Puzzle

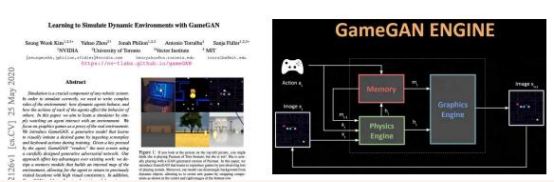
· **Stack Placement** — Players place chip stacks on adjacent hexagonal board cells
· **Color Matching** — When the top color of a placed stack matches an adjacent stack, they auto-sort
· **Clear Condition** — When 10+ same-color chips accumulate, they disappear and count toward the goal
· **Level Clear** — Fill the target score to clear the level

- As of 2026, Hexa Sort by Lion Studios has surpassed 63M downloads and $50M in revenue, with an average revenue per download of $0.80,establishing itself as the defining title of the hybrid casual puzzle genre.
- Just as Candy Crush defined the three-match puzzle genre, Hexasort paved the way for hexasort puzzles to become a standalone genre in mobile casual gaming, with multiple competing titles now in the market.

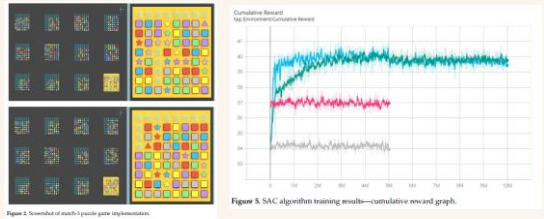


1. Project Background

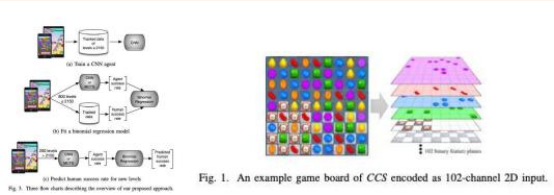
Hot Topic in Game Research : AI-Based Level Design



「Learning to Simulate Dynamic Environments with GameGAN」(Nvidia, 2020)



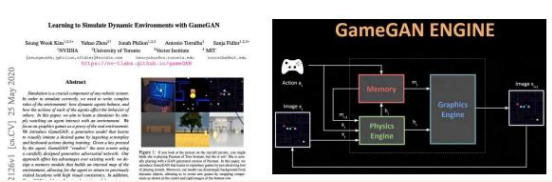
「Learning to Simulate Dynamic Environments with GameGAN」(Nvidia, 2020)



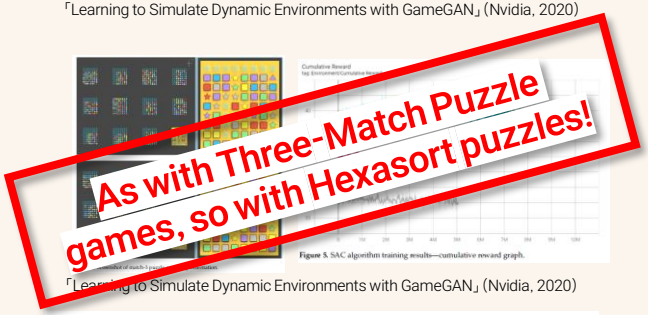
「Human-Like Playtesting with Deep Learning」(CIG 2018)

1. Project Background

Hot Topic in Game Research : AI-Based Level Design



「Learning to Simulate Dynamic Environments with GameGAN」(Nvidia, 2020)



「Learning to Simulate Dynamic Environments with GameGAN」(Nvidia, 2020)

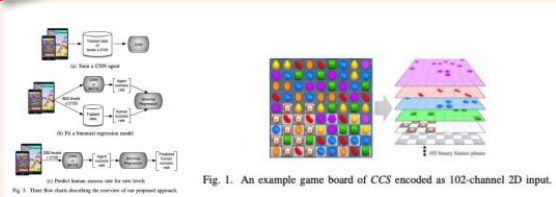
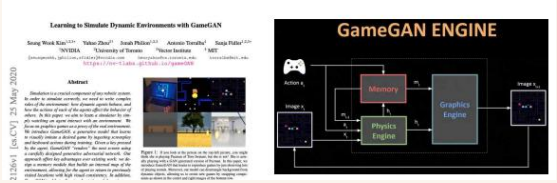


Fig. 1. An example game board of *CCS* encoded as 102-channel 2D input.

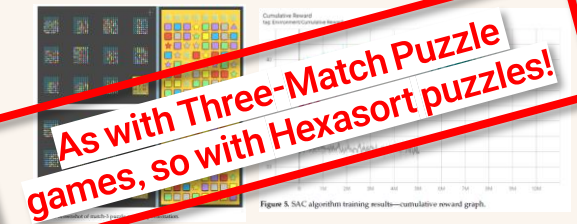
「Human-Like Playtesting with Deep Learning」(CIG 2018)

1. Project Background

Hot Topic in Game Research : AI-Based Level Design



「Learning to Simulate Dynamic Environments with GameGAN」(Nvidia, 2020)



As with Three-Match Puzzle games, so with Hexasort puzzles!

Figure 8. SAC algorithm training results—cumulative reward graph.

「Learning to Simulate Dynamic Environments with GameGAN」(Nvidia, 2020)

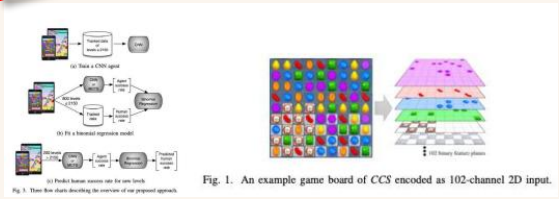
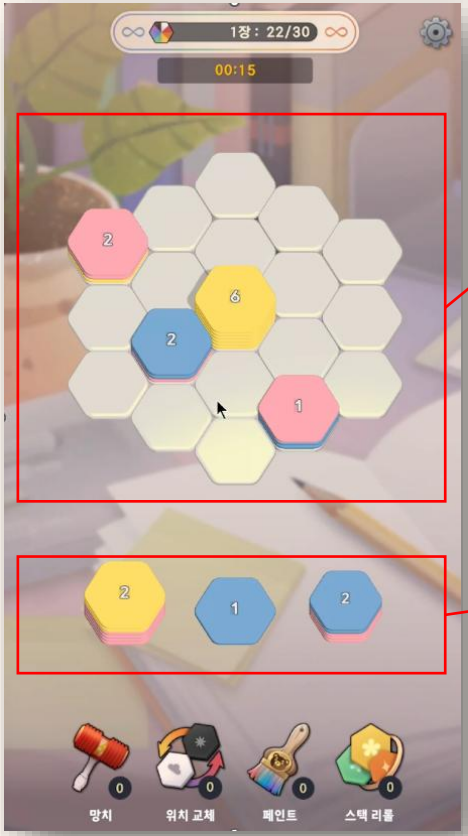


Fig. 1. An example game board of *CCS* encoded as 102-channel 2D input.

「Human-Like Playtesting with Deep Learning」(CIG 2018)

Problem 1	Solution
500 Levels Handcrafted Designers manually set each level parameter in PPT → Inefficient and error-prone	**Formula-Based Auto Generation** Auto-generates 500 JSONs using the target(N) formula
Problem 2	**Solution**
No Objective Standard Difficulty design relies solely on subjective judgment → Cannot be verified	**Objective Market Data** SKKU Lab measured Lv1~100 → 15 H1 indicators → Curve derived
Problem 3	**Solution**
Board Shape Not Verifiable Must open JSON files directly to check → No visual feedback	**Hex Grid Visualization** Plotly-based real-time board rendering + editing

2. Data-Driven Difficulty Extraction



1. Board Shape: json file

Data Collection (SKKU Game Center Lab))

1. Directly played #1 Hexasort game Lv 1~100 & collected board configurations
2. H1-1 ~ H1-15 indicator definitions: grid count·stack count·color complexity, etc.
3. Min/Max normalization: converted to 0–100 score based on market data
4. Exponential convergence curve fitting: overall trend → target(N) formula derived
5. Local_var extraction: stored 100 per-level fluctuation patterns

$$target(N) = (70 - 52 \times e^{(-N/90)}) + 3.71 + local\_var[(N-1) \bmod 100]$$

2. Stack Info: tblStage.xlsx file

Indicator	Value	Range	Ratio
TotalAllocation ↗	20	0–30	24.4%
Initial Color Count ↗	12	—	14.6%
Stack Color Count ↗	10	—	12.2%
Color Overlap Rate (higher = easier ↑) ↘	8	—	9.8%
First Add Threshold (higher = easier ↑) ↘	10	—	12.2%
Extra Color Count ↗	8	—	9.8%
Gimmick Ratio ↗	14	—	17.1%

2. Data-Driven Difficulty Extraction

Difficulty Design Goal

Set a target difficulty target(N) for each level, and design so both **board shape and gameplay parameters** together reach that difficulty.

Calculation Method

1. Calculate target difficulty target(N) for level N (exponential convergence formula)
2. Repeatedly generate board shapes (JSON) and select one where board_score ≈ target(N)
3. If board score is above target, make gameplay easier; if below, make it harder

   stack_score target = target(N) × 2 − board_score

4. The average of the two scores converges to the target difficulty

   (board_score + stack_score) / 2 ≈ target(N)

3. Streamlit App Demo

🏠 Home

- Game intro hero banner
- Gameplay video (MP4)
- 18-frame animated game tutorial

📖 Manual

- Full pipeline explanation
- File structure & roles
- Glossary (TileType · H1 · tblStage)

📊 Difficulty Analysis

- Market Data Source & Trend Charts
- Board:Gameplay weight sliders
- Stacked bar + curve comparison

🗺️ Board Viewer

- Hexa grid rendering
- Edit mode real-time difficulty
- H1 CSV export

🎲 JSON Generator

- Auto-generates 500 levels
- ZIP download
- Real-time difficulty display

🔧 Settings / Archive

- H1 · tblStage weight adjustment
- Pie chart visualization
- GitHub auto-commit

3. Streamlit App Demo

1 Check Market Data

Difficulty Analysis Tab

Review original H1 indicator table → Understand per-level complexity distribution of market game

2 Adjust Weights

Settings Tab

15 H1 sliders + 7 tblStage sliders → Verify proportions via pie chart

3 Verify Curve

Difficulty Analysis Tab

Use board:gameplay ratio slider to verify integrated difficulty converges to target(N)

4 Review & Edit Board

Board Viewer Tab

Upload generated JSON → Visualize hex grid → Individual tile editing available

5 Generate JSON

JSON Generator Tab

Select range 1~500 → Generate → Check per-level difficulty in real time → ZIP download

6 Deploy to GitHub

Archive Tab

Save settings version → GitHub commit → Auto-deployed to Streamlit Cloud

3. Streamlit App Demo

📊 Difficulty Analysis

📋 Market Data Source

H1-1~H1-15 indicator table / Per-level trend chart / SKKU Game Center Lab Lv1~100 measurements

⚖️ Weight Sliders

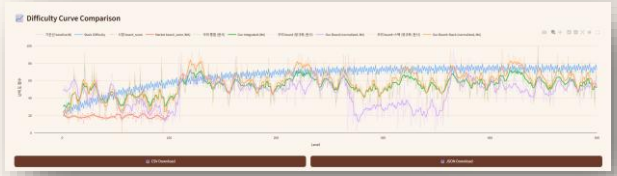
Real-time board vs gameplay ratio adjustment / Curve updates instantly on change

📈 Difficulty Curve Comparison

Our integrated difficulty vs stack difficulty (target) / Market board normalization comparison

📉 Interval Analysis

Stacked bar (board+gameplay) + integrated line + target dotted line / Per 50 levels



3. Streamlit App Demo

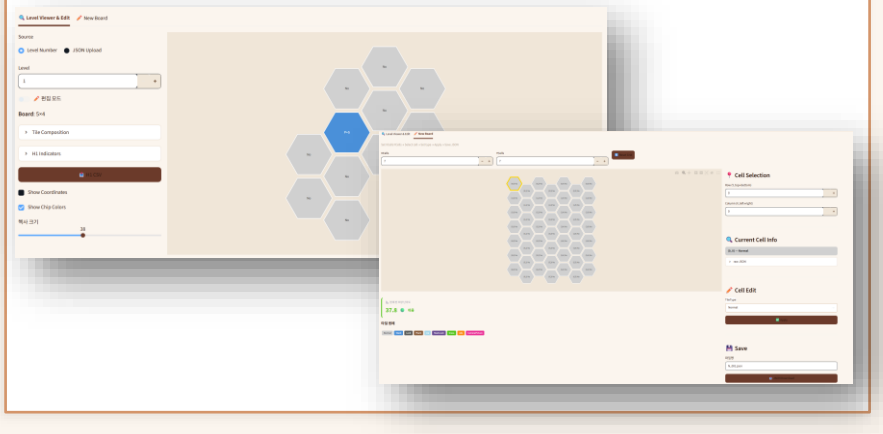
Board Viewer

Hex Rendering: Up to 5 × 4 grid, color-coded by TileType

Edit Mode: Click cell → Change TileType → H1 instantly recalculated

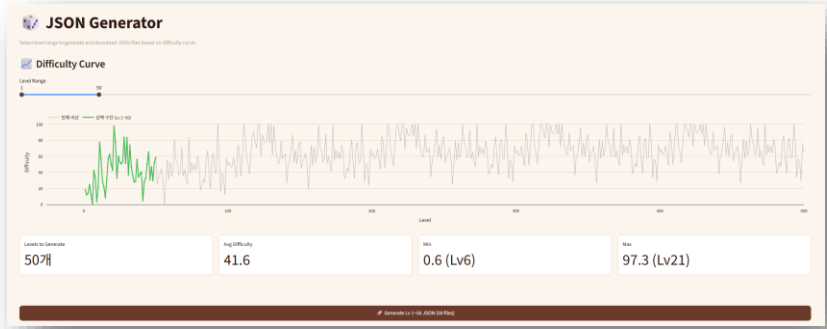
Real-time Difficulty: board_score 0–100 + grade display

H1 CSV: Export 15 indicator values



JSON Generator

1. **Level Range**
   Select range 1~500 via slider
2. **Curve Preview**
   target(N) curve + range summary
3. **Generate**
   board_score ≈ target(N) up to 10 attempts
4. **ZIP**
   Bundle N_XXX.json files for instant download
5. **GitHub**
   Overwrite upload to data/levels/



3. Streamlit App Demo

⚙️ Settings & Archive

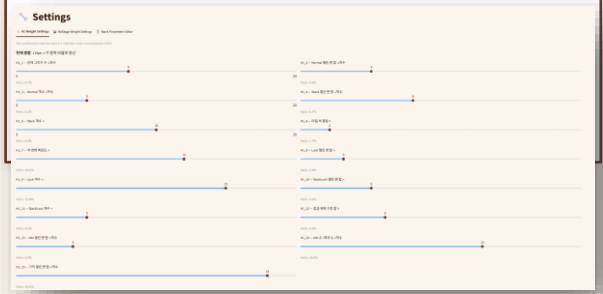
⚖️ H1 Weights

· H1-1~H1-15 sliders (0–20pt)

· Pie chart proportion visualization

· Weight change → board_score curve updates instantly

· Version management via JSON download



🎰 tblStage Weights

· 7-parameter sliders

· (TotalAlloc · color count · overlap rate · threshold, etc.)

· Pie chart + gameplay_score preview

· Heuristic-based without actual play data

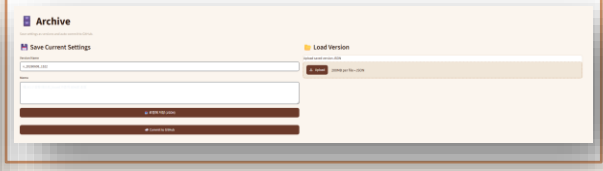
📋 Stack Parameter Editing

· Direct editing of 500 tblStage entries

· Click cell to edit

· Download as Excel after editing

· Integrated with GitHub auto-commit



4. Expected Impact & Takeaways

Project Significance

📊 Use of Real Market Data

Collected by directly playing a competing title Lv 1~100 at the SKKU Game Center Lab
→ Derived difficulty formula based on measured data, not mere assumptions

🔄 Iterative Design Structure

Post-launch play data → Recalibrate weights → Regenerate JSONs
Built-in system for continuous difficulty improvement across the game lifecycle

Reflections

What impressed me most in this project was the idea from the professor —directly implementing how difficulty is designed **in the actual game industry.** The method of weighting and summing difficulty components seems simple, but the key was knowing where to draw the baseline. Since difficulty curves themselves are not copyrightable,collecting **real play data from successful market games** as our baseline was feasible, enabling evidence-based difficulty design rather than pure intuition.

The most rewarding moment was confirming **the tool worked when applied to the actual game**. The resources for handcrafting 500 levels were dramatically reduced through automation, and I could directly feel how meaningful that is on a real development floor.

When the **professor suggested it might be worth patenting**, I became confident I had built something practical — not just a class assignment. **The cycle of recalibrating weights through regression analysis on post-launch play data and improving difficulty means,** this tool will be utilized throughout the entire game lifecycle — not a one-time use.

As someone aiming to become a level designer, this experience was a time to build the ability to handle difficulty as data — beyond simply creating levels. I believe this project will hold an important place in **my future portfolio.**

Thank You

Arts and Big Data
Shim Ahyeong · Film, TV and Multimedia
2021311066

🌐 Streamlit App: my-little-bookstorepuzzle-creator-ebmpaoaz3dupvmmmhdfm3m.streamlit.app

📦 GitHub: github.com/Ahyeong0202/My-Little-Bookstore_Puzzle-Creator